K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LINUX DEBUGGING SYSTEM

A Linux-based Command-Line Toolkit for Process, Resource and Memory Diagnostics

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

U. PRANEETH	**2520030188**
M. HARSHITH	**2520030573**
U. PRATHIK	**2520030034**

Under the esteemed guidance of

Dr. K. Anusha

Assistant Professor

Department of Computer Science and Engineering

ACADEMIC YEAR 2026-2027

DECLARATION

We hereby declare that the project report entitled “Linux Debugging System: A Linux-based Command-Line Toolkit for Process, Resource and Memory Diagnostics” is a record of the work carried out by our team, U. Praneeth (2520030188), M. Harshith (2520030573) and U. Prathik (2520030034), in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at K L (Deemed to be) University.

We further declare that the implementation and explanations presented in this report are based on the prototype developed for this project. External documentation and reference materials used for technical understanding have been acknowledged in the References chapter.

Place: Hyderabad
Date: _____

U. PRANEETH | 2520030188

Signature: _____

M. HARSHITH | 2520030573

Signature: _____

U. PRATHIK | 2520030034

Signature: _____

CERTIFICATE

This is to certify that the project report entitled “Linux Debugging System: A Linux-based Command-Line Toolkit for Process, Resource and Memory Diagnostics” is a bona fide project work carried out by U. Praneeth (2520030188), M. Harshith (2520030573) and U. Prathik (2520030034), students of the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026-2027, under my guidance.

The work presented in this report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Signature of the Guide

_____

Dr. K. Anusha
Assistant Professor

Signature of the Course Coordinator

_____

Signature of the HOD

_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. K. Anusha, Assistant Professor, Department of Computer Science and Engineering, for her guidance, encouragement and valuable suggestions throughout the development of this project.

We thank the Department of Computer Science and Engineering and K L (Deemed to be) University for providing the academic environment and laboratory facilities required to understand Linux operating-system concepts and develop the prototype.

We also thank our course faculty, classmates and friends for their feedback and support. Finally, we are grateful to our parents and family members for their encouragement during the completion of this work.

U. PRANEETH - 2520030188
M. HARSHITH - 2520030573
U. PRATHIK - 2520030034

ABSTRACT

Linux systems expose several useful interfaces for observing processes, memory, CPU activity and storage. However, a learner who is studying operating-system concepts often needs to switch between separate commands and small test programs to understand these interfaces. This project presents a menu-driven Linux Debugging System developed in the C programming language. The prototype combines a set of educational diagnostic and demonstration modules in one command-line application.

The system displays operating-system and kernel details, CPU model, uptime, RAM information, filesystem usage and process resource statistics. It inspects process information such as PID, parent PID, state, virtual memory, resident memory, thread count, command line, process group and session identifiers through the Linux /proc filesystem. It also provides a Round Robin CPU scheduling simulator that calculates completion time, turnaround time, waiting time and their averages for processes with a common arrival time of zero.

The project demonstrates inter-process communication through pipe(), fork() and waitpid(), signal delivery through sigaction() and raise(), and concurrent execution using POSIX threads protected by a mutex. A tracked-allocation module records memory blocks allocated through a project-defined wrapper and generates a text report of outstanding blocks. The tested demonstration reports two outstanding blocks totalling 320 bytes, then releases the demonstration allocations before returning.

The implementation uses GCC, standard C library functions, Linux system interfaces and POSIX threads. It is designed for Ubuntu Linux and can also be compiled in Ubuntu on Windows Subsystem for Linux. The result is an educational command-line toolkit that connects theoretical operating-system topics with observable program behaviour. Its memory tracker is intentionally limited to allocations made through the project wrapper; it is not presented as a complete replacement for a general-purpose memory-analysis tool.

Keywords: Linux, Operating System, C Programming, Process Diagnostics, /proc, Round Robin Scheduling, IPC, Signals, POSIX Threads, Mutex, Memory Leak Tracking.

TABLE OF CONTENTS

S. No.	Chapter / Topics / Subtopics	Page
	Acknowledgement / Abstract	4-5
1	Introduction	8
1.1	Background of the Project	8
1.2	Problem Statement	8
1.3	Scope of the Project	8
1.4	Objectives of the Project	8
1.5	Importance of the Application	8
1.6	Target Users / Audience	9
2	OS Concepts and Background Study	10
2.1	Operating System Concepts Related to the Project	10
2.2	Existing Methods / Algorithms	10
2.3	Proposed Approach	11
2.4	Advantages and Limitations	11
3	System Requirements	12
3.1	Hardware Requirements	12
3.2	Software Requirements	12
3.3	Operating System Environment - Ubuntu / WSL	12
3.4	Development Tools and Compilers	13
4	Technology and Tools Used	14
4.1	C Programming	14
4.2	GCC Compiler	14
4.3	Linux System Calls and APIs	14
4.4	POSIX Threads / Synchronization	14
4.5	Linux Commands and Utilities	15
4.6	Other Libraries and Interfaces	15
5	System Architecture and Design	16
5.1	Overall System Architecture	16
5.2	System Workflow	16
5.3	Flowchart / Data Flow	18
5.4	Module Description	18
5.5	Input and Output Design	19
6	Implementation	20
6.1	Module-wise Implementation	20
6.2	Algorithm / Pseudocode	20
6.3	Core C Implementation	21
6.4	Linux System Calls / APIs Used	22
6.5	Process / Thread Implementation	22
6.6	IPC / Synchronization Implementation	22
6.7	Memory Management Implementation	22
6.8	Command-line Interface Implementation	23
7	Execution and Demonstration	24
7.1	Compilation in Ubuntu	24
7.2	Execution Commands	24
7.3	Sample Inputs and Outputs	24
7.4	Execution Output Snapshots	25
7.5	Demonstration of OS Concepts	26
8	Conclusion	27
8.1	Summary of the Project	27
8.2	Objectives Achieved	27
8.3	Results Achieved	27
8.4	Skills and Learning Outcomes	27
9	References	28
9.1	Books	28
9.2	Research Papers	28
9.3	Linux / GNU Documentation	28
9.4	Online Resources	28
A	Appendix A - Source Code and Project Files	29

LIST OF FIGURES

Figure 5.1	High-level system architecture	16
Figure 5.2	Application control flow	17
Figure 5.3	Data flow for diagnostic operations	18
Figure 7.1	Round Robin sample output	25
Figure 7.2	IPC and thread output	25
Figure 7.3	Memory allocation report output	26

LIST OF TABLES

Table 2.1	OS concepts related to the project	10
Table 2.2	Advantages and limitations	11
Table 3.1	Hardware requirements	12
Table 3.2	Software requirements	12
Table 3.3	Functional and non-functional requirements	13
Table 4.1	Linux system APIs	14
Table 4.2	Linux commands and utilities	15
Table 5.1	Module descriptions	18
Table 5.2	Input and output design	19
Table 7.1	Sample scheduling inputs and results	24-25
Table 7.2	Demonstration of OS concepts	26

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

An operating system manages hardware resources and provides services that allow application programs to execute. Linux offers standard utilities and kernel interfaces for understanding running processes, memory usage, system identity and storage. Files under /proc expose information about the kernel and individual processes in a readable form. System calls and library interfaces allow a C program to create processes, exchange data, handle signals and coordinate threads.

For students, these facilities are easier to understand when they can be observed through a single small program. The Linux Debugging System is therefore developed as a command-line learning toolkit. It gathers related demonstrations behind a numbered menu and displays information in a consistent format.

1.2 Problem Statement

Operating-system concepts are commonly learned from separate lecture examples, shell utilities and independent C programs. Switching between these examples can make it difficult to connect process states, memory statistics, scheduling calculations, IPC, signals and thread synchronization. A compact educational application is needed to present these topics through repeatable inputs and visible outputs.

1.3 Scope of the Project

- Display kernel, machine, CPU, uptime, memory, filesystem and resource-usage information available to the current user.
- Inspect visible process attributes from /proc, including PID, PPID, state, memory fields, thread count, command line, process group and session.
- List a limited snapshot of processes visible in /proc and simulate Round Robin CPU scheduling for user-entered burst times.
- Demonstrate parent-child IPC using an unnamed pipe, process creation and child completion waiting.
- Demonstrate signal registration and delivery, and POSIX thread synchronization using a mutex.
- Track allocations made through the project allocator wrapper and save a human-readable memory report.

The prototype is an educational diagnostic tool. It does not attach to arbitrary processes to inspect their entire address space, does not change kernel scheduling policies, and does not automatically repair bugs in another program.

1.4 Objectives of the Project

1. Develop a menu-driven C application that compiles and executes on Ubuntu Linux.
2. Use Linux interfaces to display system, memory, filesystem and process information.
3. Implement a Round Robin simulator and calculate completion, turnaround and waiting times.
4. Illustrate process creation, pipe-based IPC and child-process synchronization.
5. Demonstrate signal handling and safe shared-counter updates using POSIX threads and a mutex.
6. Generate a report that identifies outstanding allocations made through the project tracking wrapper.

1.5 Importance of the Application

The application links theory with practice. Students can see the relationship between a process identifier and its /proc directory, observe that a parent process can communicate with a child through a pipe, and

verify how a mutex protects a shared counter. The leak demonstration also explains why allocation and deallocation must be paired and why diagnostic reports should identify the size and source location of outstanding blocks.

1.6 Target Users / Audience

- Undergraduate students studying Operating Systems and Linux system programming.
- Faculty members who need a small command-line prototype for classroom demonstrations.
- Beginner programmers learning process management, IPC, signals, threads and memory handling.
- Students preparing for OS laboratory evaluations and project demonstrations.

CHAPTER 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

OS Concept	Relationship to the Project
Process	A running program with its own process identifier and execution state. The project reads process metadata from /proc and displays PID, PPID, state and process relationships.
Process ID (PID) and Parent PID (PPID)	PID identifies a process; PPID identifies its parent. The process inspector reads these fields from the process status file.
Process groups and sessions	Related processes may be organised into process groups and sessions. The project displays PGID and SID for the selected PID.
Virtual and resident memory	VmSize describes virtual address-space size; VmRSS reports resident memory information. The Linux status interface documents these values and notes that some RSS figures are approximate [7].
CPU scheduling	The scheduler selects ready processes for CPU execution. The project simulates Round Robin scheduling with a fixed time quantum; it does not alter the kernel scheduler.
Inter-process communication	A pipe is a byte-stream communication channel. The child writes a message and the parent reads it. The pipe endpoints are closed when no longer needed [9].
Signals	A signal is an asynchronous notification delivered to a process or thread. The demonstration registers a SIGUSR1 handler and raises the signal.
Threads and synchronization	Threads share process resources. A mutex serializes access to the shared counter so concurrent updates do not overwrite one another.
Dynamic memory	Heap memory requested using malloc() should be released using free(). The project records its own wrapper allocations and reports any still outstanding at report time.

2.2 Existing Methods / Algorithms

Linux already provides utilities such as ps, top, free, df, uname and uptime. These are useful for system administration, but each focuses on a different type of information. The /proc filesystem provides process and system information in pseudo-files that applications can read [7], [8].

For source-level debugging, GCC can produce debugging information with the -g option, which is useful to a debugger such as GDB [6]. For runtime memory analysis, Valgrind Memcheck can detect several memory errors and track heap blocks that remain allocated at program termination [13]. Academic work has also described Memcheck memory-error analysis and the Valgrind dynamic binary instrumentation framework [4], [5]. The project uses these standard tools as background and learning references; its built-in leak tracker is a much smaller wrapper-based demonstration.

Round Robin is a preemptive scheduling algorithm in which each ready process receives CPU time up to a fixed quantum. Processes that are not complete return to the end of the ready queue. The prototype uses arrival time zero for all entered processes, so turnaround time equals completion time in this simulation.

2.3 Proposed Approach

1. Create one C application with a numbered command-line menu and input validation.

2. Separate the main program into focused functions for system data, process inspection, scheduling, IPC, signals, threads and memory tracking.
3. Read Linux-provided values from /proc and system APIs instead of hard-coding machine statistics.
4. Use fork(), pipe(), read(), write() and waitpid() to demonstrate parent-child communication.
5. Use sigaction() and raise() for a controlled signal demonstration.
6. Use pthread_create(), pthread_join() and mutex lock/unlock calls to coordinate shared data.
7. Maintain a linked list of tracked allocations and write outstanding allocation details to a text file.

2.4 Advantages and Limitations of the Proposed Approach

Advantages	Limitations
One menu groups several OS concepts in one application.	The prototype is intended for Linux; some /proc fields and resource values can differ by kernel and environment.
Uses standard C and commonly available Ubuntu development tools.	Some process information may be inaccessible because of permissions or process lifetime changes.
Demonstrations can be repeated with controlled inputs.	Round Robin simulation assumes all processes arrive at time zero and omits context-switch overhead.
Produces readable text reports for later inspection.	The memory tracker only sees calls made through TRACK_ALLOC; it cannot discover arbitrary malloc calls in unrelated binaries.
Shows process, IPC, signal and thread behaviour directly.	It is an educational prototype, not a production monitoring or automated repair tool.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum / Recommended Requirement
Processor	Any modern 64-bit Intel or AMD processor; dual-core or better recommended.
RAM	2 GB minimum; 4 GB or more recommended for a comfortable Ubuntu / WSL environment.
Storage	At least 500 MB free space for source, compiler tools and generated reports.
Display / Input	Standard monitor, keyboard and terminal.
Network	Not required for normal compilation or execution; internet is needed only to install packages.

3.2 Software Requirements

Software	Purpose
Ubuntu Linux	Target operating-system environment.
GCC	Compiles the C source and emits debugging information with -g [6].
GNU Make	Runs the supplied Makefile build target.
POSIX threads library	Provides thread and mutex APIs; linked with -pthread.
Terminal	Accepts menu selections and displays diagnostics.
Text editor / VS Code	Optional source-code editing.
WSL2 (optional)	Allows Ubuntu user space to run on a Windows host.

3.3 Operating System Environment - Ubuntu / WSL

The project is designed for Ubuntu Linux. It can also be built inside Ubuntu running under WSL2, provided the build tools are installed. The application uses Linux-specific /proc files and Linux system interfaces, so it should be executed from the Ubuntu terminal rather than a native Windows command prompt.

```
sudo apt update
sudo apt install -y build-essential
# Optional tools for further debugging
sudo apt install -y gdb valgrind
```

The program does not require root privileges for its normal demonstrations. However, inspecting another user's process may be restricted by system permissions or security settings. Running as root should not be necessary for the standard project demo.

3.4 Development Tools and Compilers

GCC is used to compile the single C source file. The Makefile enables common warnings, C11 language mode, no optimization for easier debugging and debug symbols. The -pthread option is used during compilation and linking for POSIX threads.

```
gcc -Wall -Wextra -Wpedantic -std=c11 -O0 -g \
    -o linux_debugging_system linux_debugging_system.c -pthread

# Or use the Makefile
make
```

3.5 Functional and Non-functional Requirements

Type	Requirement
Functional	The program shall display system identity and resource information.
Functional	The program shall inspect a PID and list a limited set of visible processes.
Functional	The program shall simulate Round Robin scheduling and calculate average waiting and turnaround times.
Functional	The program shall demonstrate pipe-based IPC, signals and synchronized threads.
Functional	The program shall report tracked allocations that have not been released.
Non-functional	The interface shall be text-based and simple to use.
Non-functional	Invalid numeric input shall be rejected and re-requested.
Non-functional	The source shall compile with warnings enabled and generate readable output files.

CHAPTER 4

TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used because it provides direct access to operating-system interfaces and is widely used for systems programming. The program uses structures, pointers, linked lists, file I/O, process identifiers, dynamic allocation and function decomposition. The project is written in standard C where possible and uses Linux/POSIX interfaces for operating-system demonstrations.

4.2 GCC Compiler

The GNU Compiler Collection compiles the source code into an executable. The -Wall, -Wextra and -Wpedantic flags request common diagnostics. The -g flag adds debugging information for use by GDB or other compatible tools [6]. The -O0 option avoids optimization during the learning build, and -pthread enables thread-related compilation and linking support.

4.3 Linux System Calls and APIs

API / Function	Purpose in the Project
uname()	Reads kernel and machine identification information.
sysinfo()	Reads uptime, memory totals and process-count information.
getrusage()	Reads resource-usage counters for the current process.
statvfs()	Reads filesystem capacity and available-space statistics.
opendir(), readdir()	Enumerates numeric process directories under /proc.
fopen(), fgets(), fread()	Reads process status, command line, CPU and memory pseudo-files.
fork()	Creates a child process for the IPC demonstration.
pipe(), read(), write(), close()	Transfers a message between parent and child through a pipe [9].
waitpid()	Waits for the child process and collects its completion status [10].
sigaction(), raise()	Registers and sends the SIGUSR1 signal.
pthread_create(), pthread_join()	Creates worker threads and waits for their completion.
pthread_mutex_lock(), pthread_mutex_unlock()	Protects the shared counter and allocation-tracker list.
malloc(), free()	Allocates and releases heap memory; project wrappers track selected allocations.

4.4 POSIX Threads / Synchronization

The thread demonstration creates two worker threads [12]. Each worker increments one shared counter 100,000 times. A mutex is locked before each increment and unlocked immediately afterwards. The main

thread joins both workers before displaying the result. The expected and actual result is 200,000, illustrating mutual exclusion and thread completion coordination.

4.5 Linux Commands and Utilities

Command	Use during setup / verification
pwd	Displays the current working directory.
ls -l	Lists the project files and permissions.
gcc --version	Checks compiler installation.
make	Builds the application using the Makefile.
./linux_debugging_system	Runs the project.
pgrep -n linux_debugging_system	Finds the latest process ID for process-inspection testing.
cat memory_leak_report.txt	Displays the generated allocation report.
cat linux_debug_report.txt	Displays the generated system snapshot.
gdb ./linux_debugging_system	Optional source-level debugging.
valgrind --leak-check=full ./linux_debugging_system	Optional external memory analysis, when Valgrind is installed.

4.6 Other Libraries and Interfaces Used

The implementation uses standard headers for file handling, string processing, dynamic memory, time and formatted output. Linux-specific headers provide system information and filesystem statistics. The pthread library supplies thread and mutex functionality. No database, web server or graphical library is required.

4.7 Proc Filesystem and File I/O

The /proc filesystem is a virtual filesystem that presents kernel and process information. The program reads /proc/meminfo for selected memory values, /proc/cpuinfo for a CPU model string, and /proc/<pid>/status and /proc/<pid>/cmdline for process details. The Linux manual documents status fields such as PID, PPID, VmSize, VmRSS and thread count [7], while /proc/meminfo reports system memory fields [8].

CHAPTER 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The architecture is divided into the user input layer, menu dispatcher, independent diagnostic modules, Linux kernel interfaces and output layer. The menu dispatcher validates a selected option and calls the corresponding function. Modules that need operating-system data read kernel-provided interfaces. The report module saves a snapshot in a text file, while the memory module writes its own allocation report.

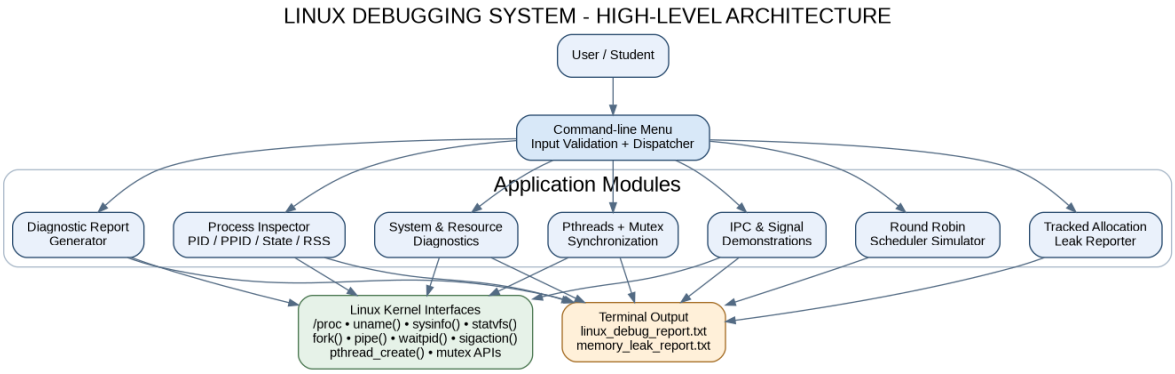


Figure 5.1: High-level architecture of the Linux Debugging System

5.2 System Workflow

After starting, the application displays a welcome message and the main menu. The user selects an operation. Numeric input is validated before a module is called. The selected module displays a result or writes a report. The program then returns to the main menu until the user chooses Exit.

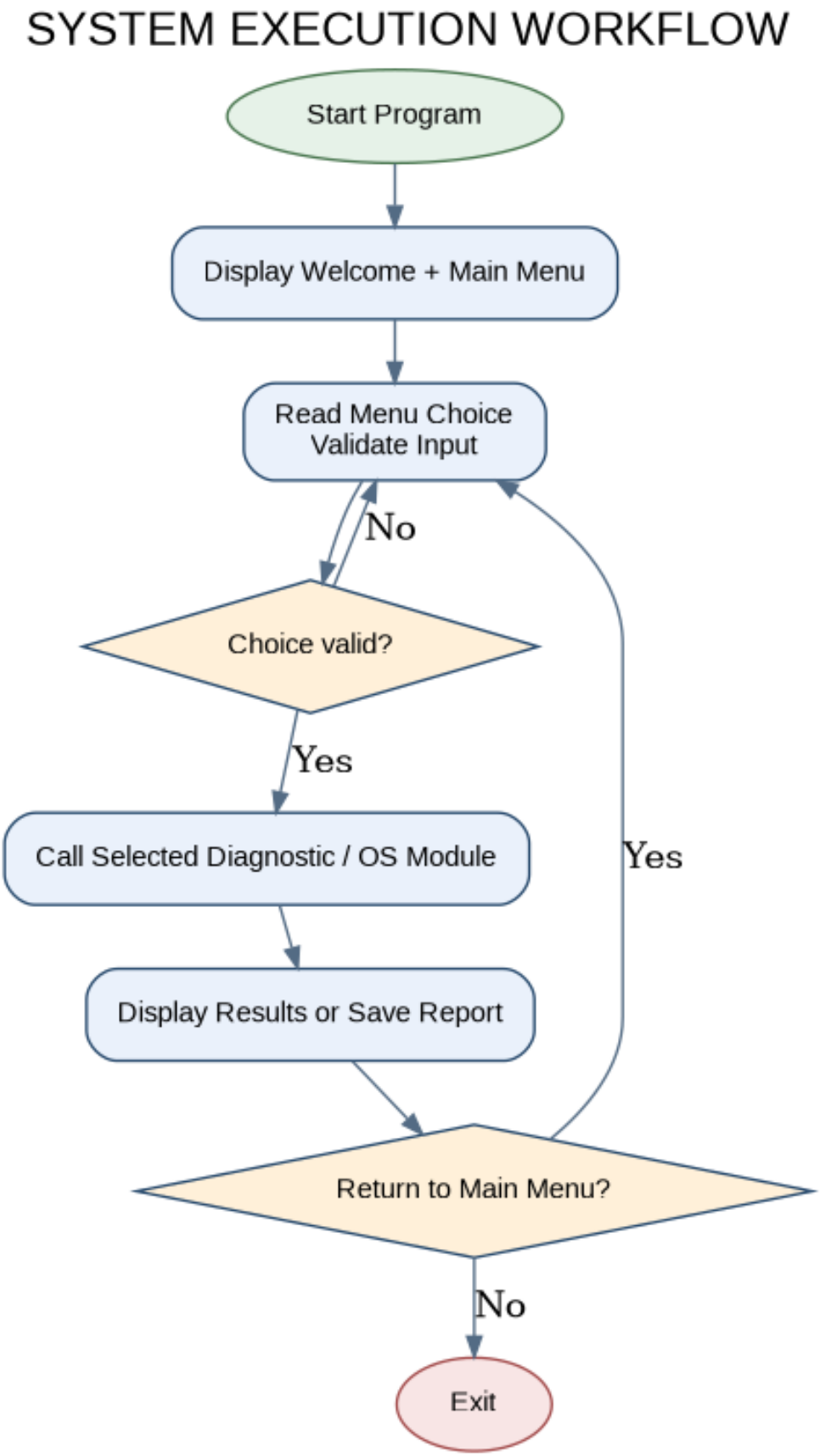


Figure 5.2: Application control flow

5.3 Flowchart / Data Flow

Input data may include a menu choice, process identifier, number of scheduling processes, burst times and time quantum. The application uses these values to select or calculate diagnostic results. Kernel information is read from /proc or system interfaces. The resulting output is displayed in the terminal or written to a report file.

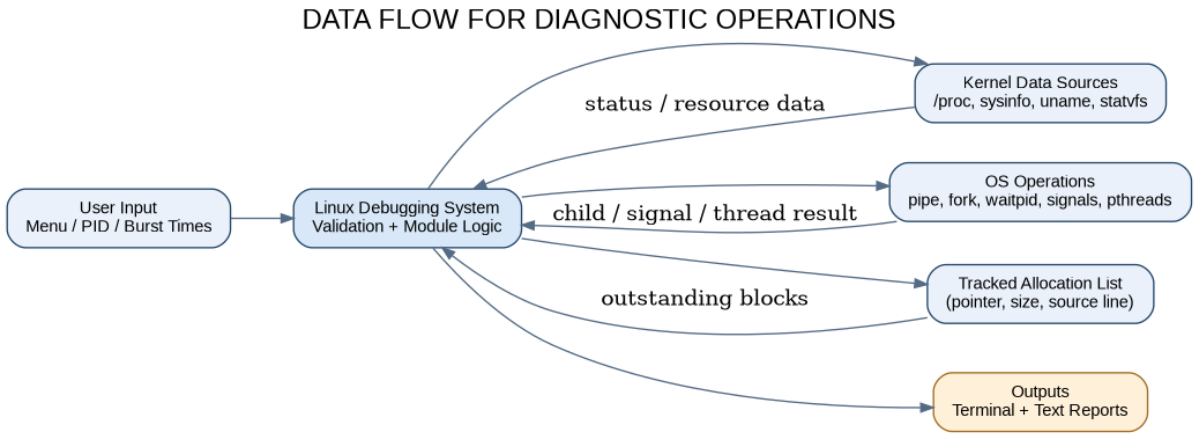


Figure 5.3: Data flow between user input, application modules, kernel interfaces and reports

5.4 Module Description

Module	Description
System overview	Displays uname, CPU model, sysinfo uptime and memory values, and getrusage counters.
Memory information	Prints selected MemTotal, MemFree, MemAvailable, Cached and swap values from /proc/meminfo.
Disk information	Uses statvfs("/") to calculate total, used and available space for the root filesystem.
Process inspector	Reads a selected /proc/<pid>/status and /proc/<pid>/cmdline and displays process IDs, state, memory, thread count, PGID and SID.
Process snapshot	Scans numeric entries under /proc and prints up to 30 visible processes with PID, PPID, state and name.
Round Robin simulator	Accepts burst times and a fixed quantum, then calculates completion, turnaround and waiting times.
IPC demonstration	Creates a pipe and child process; parent receives the child message and waits for child termination.
Signal demonstration	Installs a SIGUSR1 handler and uses raise() to deliver the signal to the current process.
Thread demonstration	Runs two worker threads and protects a shared counter with a pthread mutex.
Memory leak report	Tracks allocations created through TRACK_ALLOC and writes outstanding pointer, size and source-line details.
System report	Writes PID relationships, kernel, uptime, RAM and resource-use fields to linux_debug_report.txt.

5.5 Input and Output Design

Module	Input	Output
Process inspector	Positive numeric PID.	Name, state, PID, PPID, VmSize, VmRSS, threads, command line, PGID and SID.

Module	Input	Output
Round Robin	Process count, quantum and burst times.	Completion, turnaround and waiting times, plus averages.
IPC	No additional user input.	Parent/child PIDs, transferred message and child status.
Threads	No additional user input.	Worker count and expected/actual shared counter.
Memory tracker	No additional user input in the built-in demo.	Number and total size of outstanding tracked allocations.
Reports	Menu selection.	linux_debug_report.txt or memory_leak_report.txt.

CHAPTER 6

IMPLEMENTATION

6.1 Module-wise Implementation

The project is implemented in one C source file, linux_debugging_system.c, to keep the prototype easy to compile and present. Functions are separated by responsibility. The main() function only displays the menu, validates the selected option and dispatches control to a module. The Makefile automates compilation and cleanup.

The process inspector builds a path using the user-supplied PID and opens the status pseudo-file. The process-list module scans /proc entries and processes only directory names that contain digits. System overview and report generation use uname(), sysinfo() and getrusage(). Disk statistics are obtained through statvfs().

6.2 Algorithm / Pseudocode

Algorithm 1 - Main menu dispatcher

```
START
Display welcome message
REPEAT
    Display all menu options
    Read and validate choice
    CASE choice OF
        1: display system overview
        2: display memory information
        3: display disk information
        4: read PID and inspect process
        5: list visible processes
        6: run Round Robin simulation
        7: run pipe and fork demonstration
        8: run signal demonstration
        9: run thread and mutex demonstration
       10: run tracked memory demonstration
       11: save system diagnostic report
       12: display help
        0: exit program
    END CASE
    Wait for ENTER unless exiting
UNTIL choice = 0
STOP
```

Algorithm 2 - Round Robin scheduling (all arrival times equal zero)

```
Read number of processes N and time quantum Q
For each process i:
    Read burst[i]
    remaining[i] = burst[i]
time = 0
WHILE at least one remaining[i] > 0:
    For each process i from 1 to N:
        IF remaining[i] > 0:
            slice = MIN(Q, remaining[i])
            remaining[i] = remaining[i] - slice
            time = time + slice
            IF remaining[i] = 0:
                completion[i] = time
For each process i:
    turnaround[i] = completion[i]  // arrival time is zero
    waiting[i] = turnaround[i] - burst[i]
```

```
Calculate and display average waiting and turnaround times
```

Algorithm 3 - Tracked allocation report

```
TRACK_ALLOC(size):
    allocate memory block using malloc
    allocate metadata node
    store pointer, requested size, source file and line
    insert metadata node into allocation list under mutex
    return pointer

TRACK_FREE(pointer):
    lock allocation-list mutex
    search for matching pointer
    if found:
        unlink metadata node
        free memory block and metadata node
    unlock allocation-list mutex

REPORT:
    open memory_leak_report.txt
    lock allocation-list mutex
    write each outstanding block and accumulate count / bytes
    unlock allocation-list mutex
    write totals and close report
```

6.3 Core C Implementation

The following excerpts show the key implementation pattern. The complete source file is supplied separately in the project package.

Example: reading process status information

```
snprintf(path, sizeof(path), "/proc/%ld/status", (long)pid);
fp = fopen(path, "r");
if (!fp) {
    perror("Cannot open process status");
    return;
}
while (fgets(line, sizeof(line), fp)) {
    if (strncmp(line, "Name:", 5) == 0 ||
        strncmp(line, "State:", 6) == 0 ||
        strncmp(line, "VmRSS:", 6) == 0 ||
        strncmp(line, "Threads:", 8) == 0) {
        fputs(line, stdout);
    }
}
fclose(fp);
```

Example: thread synchronization

```
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
static long shared_counter = 0;

void *worker(void *arg) {
    long loops = *(long *)arg;
    for (long i = 0; i < loops; i++) {
        pthread_mutex_lock(&counter_mutex);
        shared_counter++;
        pthread_mutex_unlock(&counter_mutex);
    }
    return NULL;
}
```

6.4 Linux System Calls / APIs Used

The prototype uses Linux and POSIX interfaces to request information and demonstrate OS operations. File operations such as fopen() and fgets() are used to read /proc pseudo-files. fork() creates a child [11]. pipe() creates two file descriptors, and read()/write() exchange bytes. waitpid() collects child status. sigaction() installs a signal handler. pthread_create()/pthread_join() and mutex calls coordinate threads. statvfs(), uname(), sysinfo() and getrusage() provide system statistics.

6.5 Process / Thread Implementation

The process-list module scans /proc, whose numeric directory names correspond to process identifiers. For each accessible PID, it reads the process name, parent PID and state. The process inspector additionally reads memory fields, thread count and command line. It also asks the operating system for process-group and session identifiers using getpgid() and getsid().

The thread demonstration creates two independent worker threads. Both access one global counter. Since increments are shared read-modify-write operations, each update is protected by a mutex. The main thread calls pthread_join() for both workers before reading the final count. The verified test result is 200,000, matching the expected value.

6.6 IPC / Synchronization Implementation

The IPC demonstration calls pipe() before fork(). The child closes the read endpoint, writes a fixed diagnostic message to the write endpoint and exits. The parent closes the write endpoint, reads the message and calls waitpid() to wait for the child. Closing unused descriptors is important so the pipe can signal end-of-file correctly [9].

The signal module uses sigaction() to register a handler for SIGUSR1. The handler changes a volatile sig_atomic_t flag. The main function calls raise() and then checks the flag. The thread demonstration uses a mutex as the synchronization mechanism for shared state.

6.7 Memory Management Implementation

The memory tracker defines a metadata structure containing the allocated address, requested size, source file, source line and a pointer to the next record. TRACK_ALLOC calls the project allocator wrapper, allocates metadata and inserts the new record into a linked list. TRACK_FREE searches for the pointer, removes its record and releases the associated memory.

During the demonstration, blocks of 256, 128 and 64 bytes are allocated. The 128-byte block is released before the report. At report time, the 256-byte and 64-byte blocks remain in the tracking list. The generated report therefore contains two outstanding blocks and 320 outstanding bytes. The demonstration then frees those blocks so that the example does not leave memory allocated after it returns.

Scope limitation: this approach detects only allocations routed through TRACK_ALLOC. It will not automatically observe allocations made directly through malloc() elsewhere in the application or allocations in unrelated executables. A comprehensive external tool such as Valgrind Memcheck uses runtime instrumentation and tracks heap blocks more broadly [13].

6.8 Command-line Interface Implementation

The user interface is a terminal menu, not a graphical interface. read_int() validates numeric menu values and input ranges. The PID input is validated as a positive integer before a /proc path is created. Each module returns to the main menu after the user presses Enter. The help option lists the functions available in the application.

CHAPTER 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

1. Open Ubuntu terminal (or Ubuntu in WSL2).
2. Move to the folder containing linux_debugging_system.c and Makefile.
3. Install GCC and Make if they are not already installed.
4. Run make. GCC compiles the source using warnings and debug symbols.
5. Confirm that the executable linux_debugging_system is created.

```
sudo apt update
sudo apt install -y build-essential
cd linux_debugging_system_project
make
ls -l linux_debugging_system
```

7.2 Execution Commands

```
./linux_debugging_system

# In another terminal, obtain a PID for process inspection:
pgrep -n linux_debugging_system

# After option 10 or option 11, view the generated reports:
cat memory_leak_report.txt
cat linux_debug_report.txt

# Remove generated executable and reports when required:
make clean
```

7.3 Sample Inputs and Outputs

The following sample input is used to verify the Round Robin module. All arrival times are assumed to be zero.

Input	Value
Number of processes	3
Time quantum	2 time units
P1 burst time	5 time units
P2 burst time	8 time units
P3 burst time	3 time units

Process	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	5	12	12	7
P2	8	16	16	8
P3	3	11	11	8
Average			13.00	7.67

The IPC output confirms that the parent received the child message and that the child exited normally. The thread output confirms that two workers each performed 100,000 increments and the synchronized final counter was 200,000. The memory demonstration reports two tracked outstanding allocations totalling 320 bytes.

7.4 Execution Output Snapshots

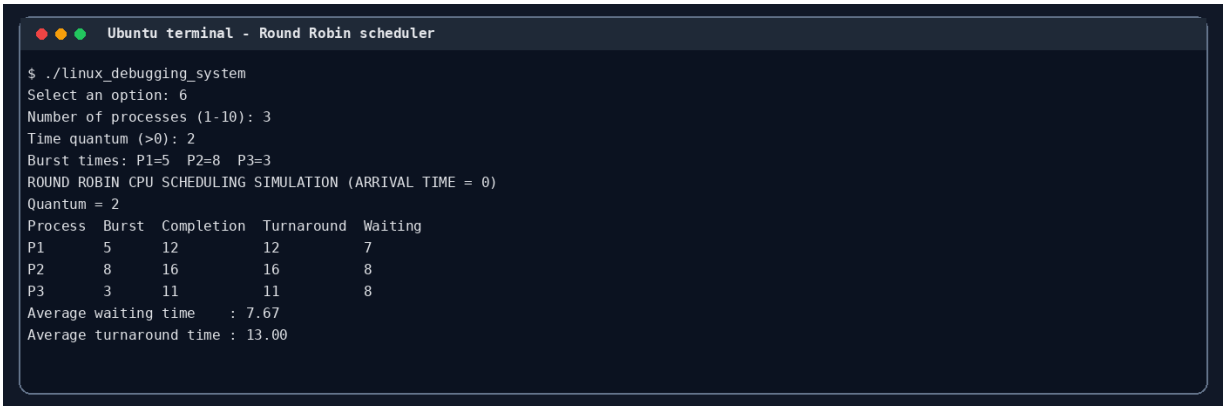


Figure 7.1: Verified Round Robin sample output (test input: 3 processes, quantum 2)

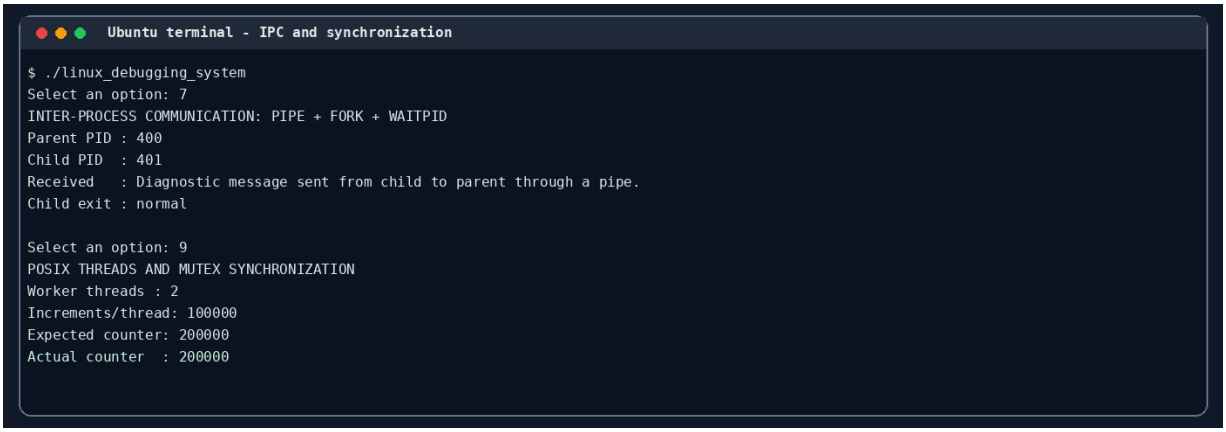


Figure 7.2: Verified pipe-based IPC and mutex-protected thread output

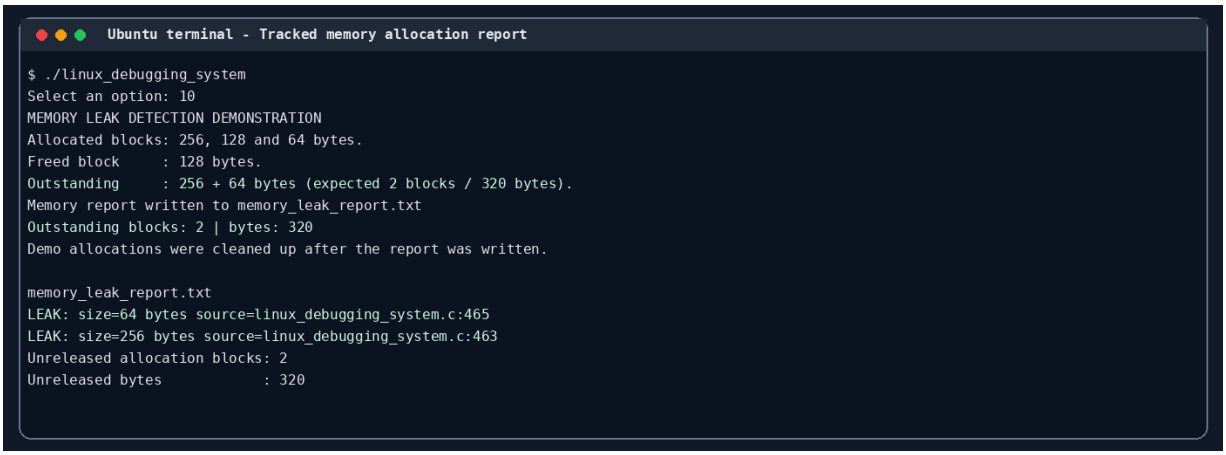


Figure 7.3: Verified tracked allocation report output

The terminal-style output snapshots above were rendered from program output captured during a local test run. Live system values such as CPU model, RAM, uptime, filesystem capacity and process identifiers will differ on the team laptop or WSL environment.

7.5 Demonstration of OS Concepts

OS Concept	Demonstration Step	Expected Observation
System information	Choose option 1.	Kernel, machine, CPU, uptime, RAM and resource counters appear.
Virtual filesystem	Choose options 2, 4 and 5.	Values and process metadata are read from /proc.
Filesystem statistics	Choose option 3.	Total, used and available root filesystem values are shown.
CPU scheduling	Choose option 6 and enter the sample values.	Completion, turnaround and waiting times are calculated.
Process creation and IPC	Choose option 7.	Child sends a message through a pipe; parent reads and waits.
Signal handling	Choose option 8.	SIGUSR1 handler executes and sets the flag.
Threads and synchronization	Choose option 9.	Expected and actual counter both equal 200,000.
Dynamic memory tracking	Choose option 10.	Two outstanding blocks / 320 bytes are written to the report.
File output	Choose option 11.	linux_debug_report.txt is created in the current directory.

CHAPTER 8

CONCLUSION

8.1 Summary of the Project

The Linux Debugging System is a menu-driven C application that combines system diagnostics and operating-system demonstrations. It reads system and process information from Linux interfaces, simulates Round Robin scheduling, demonstrates pipe-based IPC and signals, coordinates POSIX threads using a mutex, and records outstanding allocations made through a tracking wrapper.

8.2 Objectives Achieved

- Created a command-line application that compiles successfully with GCC using warning flags and debug information.
- Implemented system overview, memory, disk and process inspection modules using Linux interfaces.
- Implemented a Round Robin simulator with waiting and turnaround time calculations.
- Demonstrated a parent-child pipe message and normal child-process completion.
- Demonstrated SIGUSR1 handling and synchronized concurrent updates using POSIX threads.
- Generated text reports for tracked memory allocations and system diagnostic information.

8.3 Results Achieved

The prototype compiled without compiler warnings in the verification build. The automated menu test exercised the system-information, memory, disk, process inspection, process listing, Round Robin, IPC, signal, thread, memory report, system report and help functions. For the Round Robin test with burst times 5, 8 and 3 and quantum 2, the average waiting time was 7.67 units and the average turnaround time was 13.00 units. The IPC test transferred the expected message. The thread test reached 200,000. The leak test reported two outstanding blocks totalling 320 bytes.

8.4 Skills and Learning Outcomes

- Understanding Linux process identifiers, parent-child relationships, process groups and sessions.
- Reading system and process pseudo-files under /proc.
- Using Linux and POSIX APIs from C programs.
- Understanding CPU scheduling calculations and Round Robin execution order.
- Implementing pipes, signals, threads and mutex synchronization.
- Managing heap allocations and producing diagnostic reports.
- Compiling, testing and documenting a Linux command-line project.

8.5 Future Enhancements

- Add periodic process monitoring and CPU usage sampling over time.
- Add an interactive process tree and filters by process name or state.
- Extend scheduling simulation to FCFS, SJF, Priority and multiple arrival times.
- Add file-based logs with severity levels and timestamps.
- Add additional allocation wrappers and call-stack capture for a more detailed project-level memory report.
- Integrate optional GDB and Valgrind commands for external debugging workflows.

CHAPTER 9

REFERENCES

9.1 Books

[1] Abraham Silberschatz, Peter B. Galvin and Greg Gagne, Operating System Concepts, 10th Edition, Wiley, 2018.

[2] Michael Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

[3] Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd Edition, Prentice Hall, 1988.

9.2 Research Papers

[4] Julian Seward and Nicholas Nethercote, "Using Valgrind to Detect Undefined Value Errors with Bit-Precision," Proceedings of the 2005 USENIX Annual Technical Conference, 2005.

[5] Nicholas Nethercote and Julian Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2007, pp. 89-100.

9.3 Linux / GNU Documentation

[6] GNU Compiler Collection, "Debugging Options," https://gcc.gnu.org/onlinedocs/gcc/Debugging-Options.html (accessed 30 October 2026).

[7] Linux man-pages project, "proc_pid_status(5)," https://www.man7.org/linux/man-pages/man5/proc_pid_status.5.html (accessed 30 October 2026).

[8] Linux man-pages project, "proc_meminfo(5)," https://man7.org/linux/man-pages/man5/proc_meminfo.5.html (accessed 30 October 2026).

[9] Linux man-pages project, "pipe(7)," https://man7.org/linux/man-pages/man7/pipe.7.html (accessed 30 October 2026).

[10] Linux man-pages project, "wait(2) / waitpid(2)," https://man7.org/linux/man-pages/man2/waitpid.2.html (accessed 30 October 2026).

[11] Linux man-pages project, "fork(2)," https://www.man7.org/linux/man-pages/man2/fork.2.html (accessed 30 October 2026).

[12] Linux man-pages project, "pthreads(7)," https://man7.org/linux/man-pages/man7/pthreads.7.html (accessed 30 October 2026).

9.4 Online Resources

[13] Valgrind Documentation, "Memcheck: a memory error detector," https://valgrind.org/docs/manual/mc-manual.html (accessed 30 October 2026).

[14] Valgrind, "Research Papers," https://valgrind.org/docs/pubs.html (accessed 30 October 2026).

[15] GNU Make Manual, https://www.gnu.org/software/make/manual/ (accessed 30 October 2026).

CHAPTER A

APPENDIX A - SOURCE CODE AND PROJECT FILES

A.1 Project Directory Structure

```
linux_debugging_system_project/
|-- linux_debugging_system.c
|-- Makefile
|-- README.md
|-- memory_leak_report.txt       (generated after option 10)
|-- linux_debug_report.txt       (generated after option 11)
|-- report_images/               (report figures / output captures)
```

A.2 Compilation and Execution Summary

```
make
./linux_debugging_system

# To remove generated outputs and executable:
make clean
```

A.3 Important Implementation Notes

- The menu application is terminal based; no GUI is required.
- The process-list module displays at most 30 processes to keep output readable.
- The Round Robin simulator assumes the same arrival time (zero) for all processes.
- The memory leak demonstration intentionally creates two tracked outstanding blocks for its report and then frees them before returning.
- The source file and README are provided as separate files in the accompanying project package.